

软件模式

1. 软件模式是将模式的一般概念应用于软件开发领域，即软件开发的总体指导思路或参照样板。软件模式并非仅限于设计模式，还包括架构模式、分析模式和过程模式等，实际上，在软件生存期的每一个阶段都存在着一些被认同的模式。
2. 软件模式可以认为是对软件开发这一特定“问题”的“解法”的某种统一表示，软件模式等于一定条件下的出现的问题以及解法。软件模式的基础结构由 4 个部分构成：问题描述、前提条件（环境或约束条件）、解法和效果。
3. 要能讲清为什么使用某个模式
4. 软件模式与具体的应用领域无关，在模式发现过程中需要遵循大三律（Rule of Three），即只有经过三个以上不同类型（或不同领域）的系统的校验，一个解决方案才能从候选模式升格为模式

设计模式的定义

设计模式（Design Pattern）是一套被反复使用、多数人知晓的、经过分类编目的、代码设计经验的总结，使用设计模式是为了可重用代码、让代码更容易被他人理解、保证代码可靠性。

设计模式的基本要素

设计模式一般有如下几个基本要素：模式名称、问题、目的、解决方案、效果、实例代码和相关设计模式，其中的关键元素包括以下四个方面：

1. 模式名称（Pattern name）
2. 问题（Problem）
3. 解决方案（Solution）
4. 效果（Consequences）

设计模式的分类

1. 根据其目的（模式是用来做什么的）可分为创建型（Creational），结构型（Structural）和行为型（Behavioral）三种：
 1. 创建型模式主要用于创建对象。
 2. 结构型模式主要用于处理类或对象的组合。
 3. 行为型模式主要用于描述对类或对象怎样交互和怎样分配职责。
2. 根据范围，即模式主要是用于处理类之间关系还是处理对象之间的关系，可分为类模式和对象模式两种：
 1. 类模式处理类和子类之间的关系，这些关系通过继承建立，在编译时刻就被确定下来，是属于静态的。
 2. 对象模式处理对象间的关系，这些关系在运行时刻变化，更具动态性

享元模式为什么是结构型模式：

模板方法为什么是行为型模式：子类控制父类的行为

设计模式与类库框架

设计原则

1. 对于面向对象的软件系统设计来说，在支持可维护性的同时，需要提高系统的可复用性。
2. 软件的复用可以提高软件的开发效率，提高软件质量，节约开发成本，恰当的复用还可以改善系统的可维护性。
3. 单一职责原则：要求在软件系统中，一个类只负责一个功能领域中的相应职责。
4. 开闭原则：要求一个软件实体应当对扩展开放，对修改关闭，即在不修改源代码的基础上扩展一个系统的行为。
5. 里氏代换原则：可以通俗表述为在软件中如果能够使用基类对象，那么一定能够使用其子类对象。
6. 依赖倒转原则：要求抽象不应该依赖于细节，细节应该依赖于抽象；要针对接口编程，不要针对实现编程。

- 7. 接口隔离原则：要求客户端不应该依赖那些它不需要的接口，即将一些大的接口细化成一些小的接口供客户端使用。
- 8. 合成复用原则：要求复用尽量使用对象组合，而不使用继承。
- 9. 迪米特法则：要求一个软件实体应当尽可能少的与其他实体发生相互作用。

设计原则之间的关系

- 目标：开闭原则
- 指导：迪米特法则（最小知识原则）
- 基础：单一职责原则、可变性封装原则
- 实现：依赖倒转原则、合成复用原则、里氏代换原则、接口隔离原则

模式的别名

模式名称	模式英文名	模式别名
简单工厂模式	Simple Factory Pattern	静态工厂方法（Static Factory Method）模式
工厂方法模式	Factory Method Pattern	工厂模式，虚拟构造器（Virtual Constructor）模式或者多态工厂（Polymorphic Factory）模式
抽象工厂模式	Abstract Factory Pattern	Kit 模式
策略模式	Strategy Pattern	Policy Pattern
状态模式	State Pattern	状态对象（Objects for States）
命令模式	Command Pattern	动作（Action）模式、事务（Transaction）模式
适配器模式	Adapter Pattern	包装器（Wrapper）
装饰器模式	Decorator Pattern	包装器（Wrapper）、油漆工模式（翻译不同）
中介者模式	Mediator Pattern	调停者模式（翻译不同）
桥接模式	Bridge Pattern	柄体（Handle and Body）模式、接口（Interface）模式
外观模式	Facade Pattern	门面模式
享元模式	Flyweight Pattern	轻量级模式

练习题

- 1. Every Design Pattern allows the developer to vary different parts of a design. Complete the following table:

Purpose	Design Pattern	可变的方面
创建型模式	抽象工厂模式	容易添加新的具体工厂类、新的产品层级、产品族和具体产品
结构型模式	适配器模式	c
结构型模式	装饰者模式	容易添加被装饰的组件、添加的额外职责和装饰后的组件
行为型模式	观察者模式	容易添加具体的观察者和被观察者，以及他们之间的观察关系
行为型模式	状态模式	方便地增加新的状态，只需要改变对象状态即可改变对象的行为
创建型模式	工厂方法模式	容易添加新的具体产品和新的具体工厂
行为型模式	策略模式	容易添加新的策略
结构型模式	外观模式	不容易修改：在不引入抽象外观类的情况下，增加新的子系统可能需要修改外观类或客户端的源代码，违背了“开闭原则”
行为型模式	命令模式	容易添加新命令、设计组合命令和宏命令、方便实现命令的 undo 和 redo

1. In object-oriented programming one is advised to avoid case (and if) statements. Select one design pattern that helps avoid case statements and explain how it helps.

策略模式：将不同的策略逻辑放入不同的策略类中，而不是放到一个大的 if-else 或 case 语句中管理，实现了逻辑控制代码和实现代码的分离，客户类只需要设置好策略后进行类似 `strategy.operate()` 的调用即可完成操作。

1. The ATM provides two types of transaction, withdrawals and deposits. For every transaction, the user specifies a reference account (for deposit to or withdrawal from) and a dollar amount. Withdrawals can either be made in cash to the user or as a transfer to another account. For cash withdrawals, the user chooses the denominations of the payment; for transfers, the user specifies a target account. Draw the design class diagram. Code is not required.

2. 软件详细设计简答题

【2017】请至少说出三个面向对象的原则，并解释它们如何应用于策略模式？ Please name at least three Object-Oriented principles, and explain how they are applied in Strategy pattern?

1. 单一职责原则
2. 开闭原则
3. 里氏代换原则
4. 合成复用原则

【2019】设计模式是什么？举例说明类模式和对象模式的区别？

1. 什么是设计模式：
 1. 设计模式是对被用来在特定场景下解决一般设计问题的类和互相通信的对象的描述，包括模式名称、问题、解决方案和效果四部分。

2. 设计模式 (Design Pattern) 是一套被反复使用、多数人知晓的、经过分类编目的、代码设计经验的总结, 使用设计模式是为了可重用代码、让代码更容易被他人理解、保证代码可靠性。

2. 设计模式分类

1. 根据其目的 (模式是用来做什么的) 可分为创建型 (Creational), 结构型 (Structural) 和行为型 (Behavioral) 三种:
 1. 创建型模式主要用于创建对象。
 2. 结构型模式主要用于处理类或对象的组合。
 3. 行为型模式主要用于描述对类或对象怎样交互和怎样分配职责。
2. 根据范围, 即模式主要是用于处理类之间关系还是处理对象之间的关系, 可分为类模式和对象模式两种:
 1. 类模式处理类和子类之间的关系, 这些关系通过继承建立, 在编译时刻就被确定下来, 是属于静态的。
 2. 对象模式处理对象间的关系, 这些关系在运行时刻变化, 更具动态性。

【2019】防御式编程是什么? 断言和错误处理的区别?

1. 可以预见到 (至少预先推测到) 问题所在, 断定代码中每个阶段可能出现的错误, 并作出相应的防范措施, 来防止类似的意外的发生。
2. 断言和错误处理的区别
 1. 断言是在开发期间使用的、让程序在运行时进行自检的代码, 是对开发人员的警告, 通常是一个子程序或宏。断言不可以有副作用。
 2. 错误处理是对预先已经考虑到的错误 (如用户错误、程序错误、意外情况等等) 按照流程进行处理。

【2019】设计模式对 MVC 的影响?

1. MVC 模式使用了运行时、动态和相互之间的关系集成到开发框架中, 是分层模式的变种。
 1. 分为 model (业务逻辑)、view (处理用户展示, 接收用户操作)、controller (对用户操作进行处理, 将信息通知给 model) (强调模块间约束关系, model 不可以直接返回到 controller)
 2. 优点: 耦合性低, 重用性高, 生命周期成本低, 部署快, 可维护性高, 方便管理
 3. 缺点: 没有明确定义, 不适用于中小型应用程序, 增加实现复杂度, 视图和控制器过于紧密, 视图对模型访问低效。
2. 四人书中, MVC 模式是观察者模式、策略模式和组合模式的演化, 可能涉及到工厂模式和装饰器模式
 1. 基于推送-订阅模式
 2. 观察者: model 发生变化通知 controller, 然后更新 view
 3. 策略模式: controllers 帮助 views 对不同用户的输入做不同的响应。
 4. 组合模式: 一组 views

【2019】策略模式和状态模式的区别?

1. 在状态模式中, 具体状态类的方法参数中包含上下文对象, 需要在状态处理完成后完成状态切换。
2. 在策略模式中, 直接对上下文类调用 set 方法设置策略即可, 不涉及到策略的切换。

【2019】最小知识原则在设计模式中的应用?

1. 中介者模式
2. 外观模式

【2022】 Please explain the Liskov Substitution Principle and how it contributes to the Open- Closed Principle. (6 points)

【2022】 Please explain how the Factory Method Pattern and Abstract Factory Pattern adhere to the Open-Closed Principle. (6 points)

【2022】 What is the benefit of decoupling the Receiver from the Invoker in the Command Pattern? (6 points)

4. There are two methods for propagating data to observers with the Observer design pattern: the push model and the pull model. Why would one model be preferable over the other? What are the trade-offs of each model? (6 points)

【2022】 What is the difference between the categories of Creational Patterns and Structural Patterns? What categories do Prototype and Flyweight fall into, respectively? Explain why. (6 points)

一个游戏，有几种人类角色：骑士、骑兵、步兵，持有不同的武器（矛、剑、斧），拥有 fight 方法；有一个非人类角色巨魔，可以持有武器，但攻击方法不同（beat）。现在希望让巨魔和其他人类角色一起进行游戏，并且要求有角色死亡时其他活着的角色要收到通知。运用设计模式进行设计，并画出类图。

描述	模式
Wraps an object and provides a different interface to it.	适配器模式
Subclasses decide how to implement steps in an algorithm.	模板方法模式
Subclasses decide which concrete classes to create.	工厂方法模式
Ensures one and only one object is created.	单例模式
Encapsulates interchangeable behaviors and use delegation to decide which one to use.	策略模式
Clients treat collections of objects and individual objects uniformly.	组合模式
Encapsulates state-based behaviors and use delegation to switch between behaviors.	状态模式
Provides a way to traverse a collection of objects without exposing its implementation.	迭代器模式？
Simplifies the interface of a set of classes.	外观模式
Wraps an object to provide new behavior.	装饰模式
Allows objects to be notified when state changes.	观察者模式
Encapsulate a request as an object.	命令模式